2.4 Running the build

Now that you have added some files and content, let's make a first build of the docs. A build is started with the **sphinx-build** program:

```
$ sphinx-build -M html sourcedir outputdir
```

where *sourcedir* is the *source directory*, and *outputdir* is the directory in which you want to place the built documentation. The *-M* option selects a builder; in this example Sphinx will build HTML files.

See also

Refer to the *sphinx-build man page* for all options that **sphinx-build** supports.

You can also build a **live version of the documentation** that you can preview in the browser. It will detect changes and reload the page any time you make edits. To do so, use sphinx-autobuild[27] to run the following command:

```
$ sphinx-autobuild source-dir output-dir
```

However, **sphinx-quickstart** script creates a `Makefile` and a `make.bat` which make life even easier for you. These can be executed by running **make** with the name of the builder. For example.

```
$ make html
```

This will build HTML docs in the build directory you chose. Execute **make** without an argument to see which targets are available.

How do I generate PDF documents?

`make latexpdf` runs the *LaTeX builder* and readily invokes the pdfTeX toolchain for you.

2.5 Documenting objects

One of Sphinx's main objectives is easy documentation of *objects* (in a very general sense) in any *domain*. A domain is a collection of object types that belong together, complete with markup to create and reference descriptions of these objects.

The most prominent domain is the Python domain. For example, to document Python's built-in function `enumerate()`, you would add this to one of your source files.

```
.. py:function:: enumerate(sequence[, start=0])

   Return an iterator that yields tuples of an index and an item of the
   *sequence*. (And so on.)
```

This is rendered like this:

enumerate(*sequence*[, *start=0*])

Return an iterator that yields tuples of an index and an item of the *sequence*. (And so on.)

The argument of the directive is the *signature* of the object you describe, the content is the documentation for it. Multiple signatures can be given, each in its own line.

[27] https://github.com/sphinx-doc/sphinx-autobuild

`gettext_location`

Type
bool

Default
True

If True, Sphinx generates location information for messages in message catalogs.

Added in version 1.3.

`gettext_auto_build`

Type
bool

Default
True

If True, Sphinx builds a `.mo` file for each translation catalog file.

Added in version 1.3.

`gettext_additional_targets`

Type
set[str] | Sequence[str]

Default
[]

Enable `gettext` translation for certain element types. Example:

```
gettext_additional_targets = {'literal-block', 'image'}
```

The following element types are supported:

- `'index'` – index terms
- `'literal-block'` – literal blocks (`::` annotation and `code-block` directive)
- `'doctest-block'` – doctest block
- `'raw'` – raw content
- `'image'` – image/figure uri

Added in version 1.3.

Changed in version 4.0: The alt text for images is translated by default.

Changed in version 7.4: Permit and prefer a set type.

`figure_language_filename`

Type
str

Default
'{root}.{language}{ext}'

The filename format for language-specific figures. The available format tokens are:

- `{root}`: the filename, including any path component, without the file extension. For example: `images/filename`.

Added in version 5.2.

Options for source files

exclude_patterns

Type
Sequence[str]

Default
()

A list of *glob-style patterns* that should be excluded when looking for source files. They are matched against the source file names relative to the source directory, using slashes as directory separators on all platforms. *exclude_patterns* has priority over *include_patterns*.

Example patterns:

- 'library/xml.rst' – ignores the library/xml.rst file
- 'library/xml' – ignores the library/xml directory
- 'library/xml*' – ignores all files and directories starting with library/xml
- '**/.git' – ignores all .git directories
- 'Thumbs.db' – ignores all Thumbs.db files

exclude_patterns is also consulted when looking for static files in *html_static_path* and *html_extra_path*.

Added in version 1.0.

include_patterns

Type
Sequence[str]

Default
('**',)

A list of *glob-style patterns* that are used to find source files. They are matched against the source file names relative to the source directory, using slashes as directory separators on all platforms. By default, all files are recursively included from the source directory. *exclude_patterns* has priority over *include_patterns*.

Example patterns:

- '**' – all files in the source directory and subdirectories, recursively
- 'library/xml' – just the library/xml directory
- 'library/xml*' – all files and directories starting with library/xml
- '**/doc' – all doc directories (this might be useful if documentation is co-located with source files)

Added in version 5.1.

master_doc

root_doc

Type
str

Default
'index'

Added in version 1.8.

Changed in version 3.5: Support the *priority* attribute

html_js_files

Type
Sequence[str | tuple[str, dict[str, str]]]

Default
[]

A list of JavaScript files. The entry must be a *filename* string or a tuple containing the *filename* string and the *attributes* dictionary. The *filename* must be relative to the *html_static_path*, or a full URI with scheme like 'https://example.org/script.js'. The *attributes* dictionary is used for the <script> tag's attributes.

Example:

```
html_js_files = [
    'script.js',
    'https://example.com/scripts/custom.js',
    ('custom.js', {'async': 'async'}),
]
```

As a special attribute, *priority* can be set as an integer to load the JavaScript file at an earlier or later step. For more information, refer to *Sphinx.add_js_file()*.

Added in version 1.8.

Changed in version 3.5: Support the *priority* attribute

html_static_path

Type
list[str]

Default
[]

A list of paths that contain custom static files (such as style sheets or script files). Relative paths are taken as relative to the *configuration directory*. They are copied to the output's _static directory after the theme's static files, so a file named default.css will overwrite the theme's default.css.

As these files are not meant to be built, they are automatically excluded from source files.

Note

For security reasons, dotfiles under html_static_path will not be copied. If you would like to copy them intentionally, explicitly add each file to this setting:

```
html_static_path = ['_static', '_static/.htaccess']
```

An alternative approach is to use *html_extra_path*, which allows copying dotfiles under the directories.

autodoc will continue to look for multiple signature lines, stopping at the first line that does not look like a signature. This is useful for declaring overloaded function signatures.

Added in version 1.1.

Changed in version 3.1: Support overloaded signatures

Changed in version 4.0: Overloaded signatures do not need to be separated by a backslash

autodoc_mock_imports

Type
list[str]

Default
[]

This value contains a list of modules to be mocked up. This is useful when some external dependencies are not met at build time and break the building process. You may only specify the root package of the dependencies themselves and omit the sub-modules:

```
autodoc_mock_imports = ['django']
```

Will mock all imports under the `django` package.

Added in version 1.3.

Changed in version 1.6: This config value only requires to declare the top-level modules that should be mocked.

autodoc_typehints

Type
str

Default
'signature'

This value controls how to represent typehints. The setting takes the following values:

- `'signature'` – Show typehints in the signature
- `'description'` – Show typehints as content of the function or method The typehints of overloaded functions or methods will still be represented in the signature.
- `'none'` – Do not show typehints
- `'both'` – Show typehints in the signature and as content of the function or method

Overloaded functions or methods will not have typehints included in the description because it is impossible to accurately represent all possible overloads as a list of parameters.

Added in version 2.1.

Added in version 3.0: New option `'description'` is added.

Added in version 4.1: New option `'both'` is added.

autodoc_typehints_description_target

Type
str

Default
'all'